

DESIGN AND VERIFICATION OF 8-BIT ALU AND 4-BIT REGISTER USING RTL MODELING

USING RTL MODELING
4-BIT REGISTER

BY :

Name : BANDI THANISH KUMAR

College : IIIT DHARWAD

College : IIIT DHARWAD

Name : BANDI THANISH KUMAR

BY :

WHAT IS RTL?

RTL(Register Transfer Level) is a design level in digital electronics where we describe:

- How data moves between **registers**
- What operations are performed on that data
- All based on a **clock signal**

Simple Meaning:

It explains who sends data, who receives data, and what operation happens in between.

Example:

If $R1 \leftarrow R2 + R3$

It means:

- Data from register R2 and R3
- Goes to ALU
- Gets added
- Result stored in R1

That description is called **RTL description**

WHAT IS ALU ?(COMBINATIONAL CIRCUIT)

ALU (Arithmetic Logic Unit) is a part of the CPU that performs:

- Arithmetic operations
- Logical operations

It Performs:

- ✓ Addition
- ✓ Subtraction
- ✓ Multiplication (basic systems)
- ✓ AND
- ✓ OR
- ✓ NOT
- ✓ XOR
- ✓ Comparisons

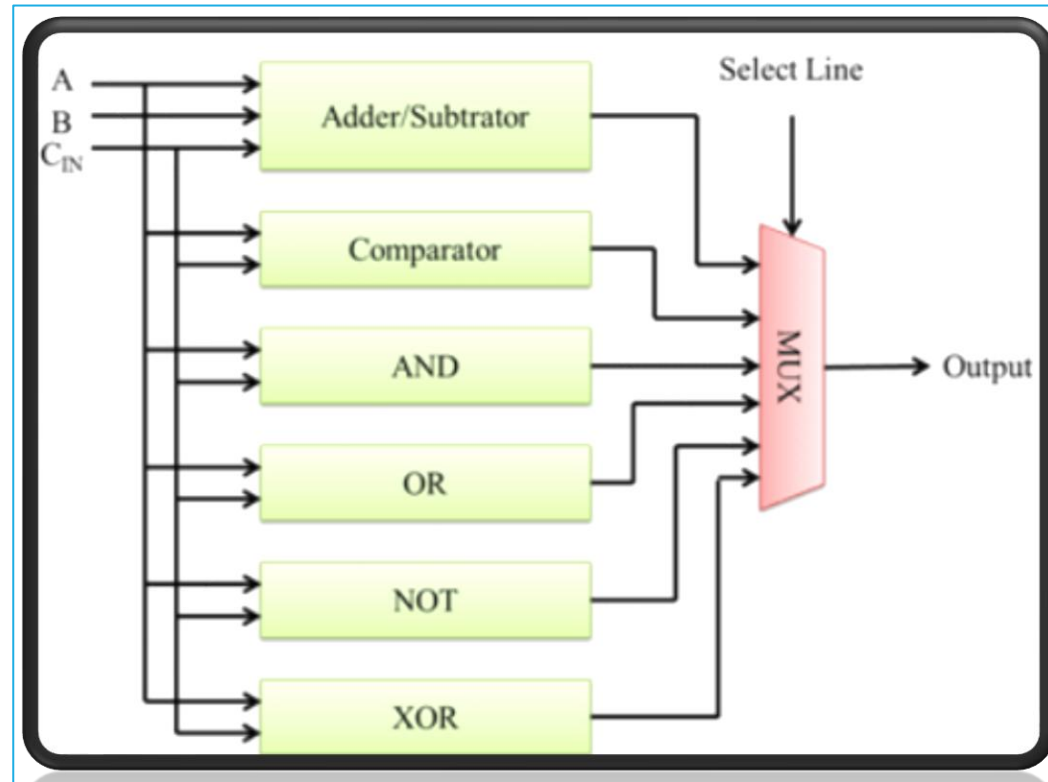
Example:

If you calculate:

$$5 + 3$$

The ALU performs this addition.

ALU is the brain for calculations inside a computer.



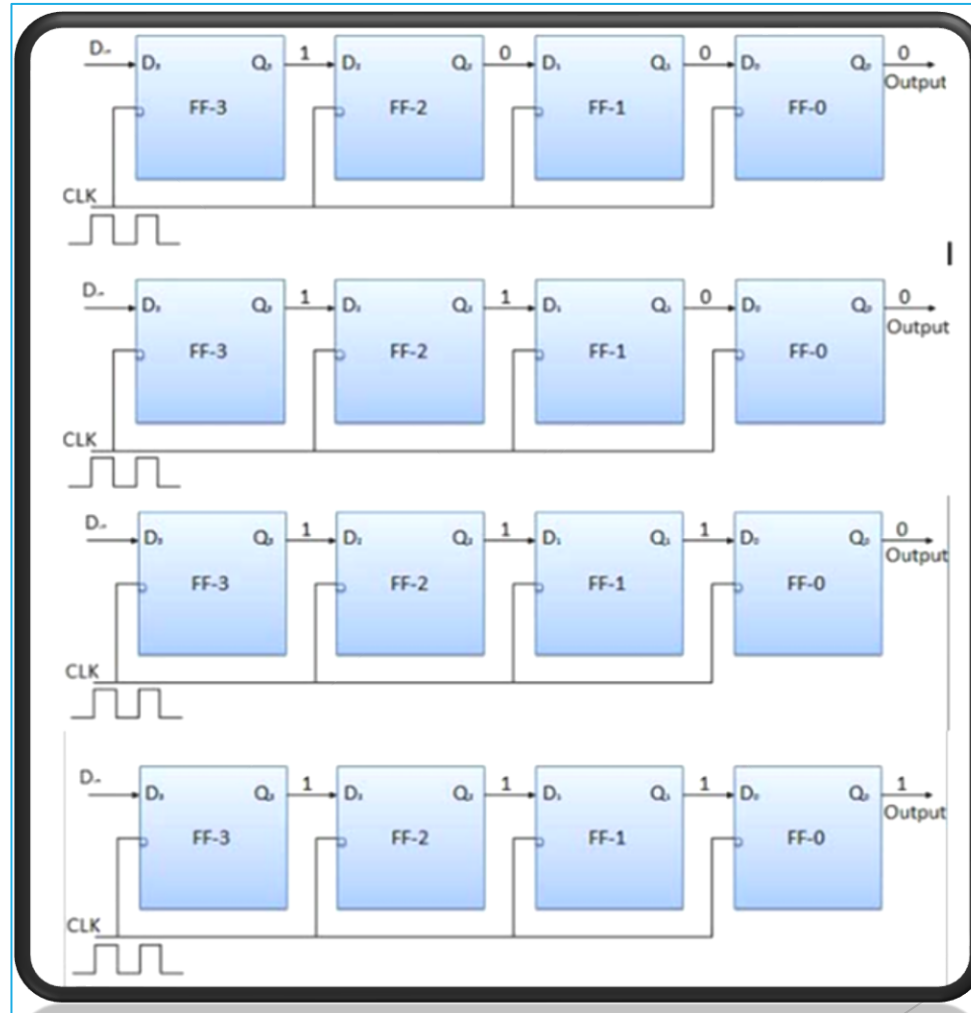
WHAT IS A 4-BIT REGISTER ? (SEQUENTIAL CIRCUIT)

A **4-BIT Register** is a type of register that:

- Stores binary data using 4 D Flip Flops
- It stores data on Rising edge of the Clock signal.
- One bit at a time
- On each clock pulse

Types :

- ✓ Serial In Serial Out (SISO)
- ✓ Serial In Parallel Out (SIPO)
- ✓ Parallel In Serial Out (PISO)
- ✓ Parallel In Parallel Out (PIPO)



INTRODUCTION

INTRODUCTION

WHAT IS RTL DESIGN (REGISTER TRANSFER LEVEL DESIGN) ?

RTL Design is the method of designing digital circuits by describing:

- Data stored in **registers**
- Transfer of data between registers
- Operations performed on the data (using ALU / logic)
- Controlled by a **clock signal**

It is usually written using HDL languages like Verilog or VHDL.

Simple Idea

RTL design tells what hardware should do every clock cycle

Example (Verilog-like idea)

```
CODE : always @(posedge clk)  
        R1 <= R2 + R3;
```

Meaning:

- On each clock pulse
- Add R2 and R3
- Store result in R1

So RTL is the **blueprint of hardware behavior** before real circuit is built.

IMPORTANCE OF SIMULATION IN DIGITAL DESIGN

Simulation = Testing the circuit in software before building real hardware.

Why it is VERY important:

1. Find errors early

Detect logic mistakes before fabrication

2. Saves money

Hardware fabrication is expensive — fixing after manufacturing is costly

3. Check functionality

Verify outputs for all input combinations

4. Timing verification

Ensures circuit works correctly with clock

5. Debugging becomes easy

Waveforms show where signal is wrong

6. Prevents hardware damage

Wrong voltage or connections can destroy chips

7. Improves reliability

Ensures design works in all cases

DIFFERENCES BETWEEN COMBINATIONAL CIRCUIT AND SEQUENTIAL CIRCUIT

FEATURE	COMBINATIONAL CIRCUIT	SEQUENTIAL CIRCUIT
DEFINITION	OUTPUT DEPENDS ONLY ON PRESENT INPUT	OUTPUT DEPENDS ON PRESENT INPUT AND PAST HISTORY
MEMORY	NO MEMORY	HAS MEMORY (STORES PREVIOUS STATE)
CLOCK	DOES NOT REQUIRE CLOCK	USUALLY REQUIRES CLOCK SIGNAL
OUTPUT CHANGE	CHANGES IMMEDIATELY WITH INPUT	CHANGES AT SPECIFIC TIME (CLOCK PULSE)
EXAMPLES	ADDER,SUBTRACTOR,MUX,ENCODER ,DECODER	FLIP FLOP,REGISTER,COUNTER,SHIFT REGISTER

PROJECT OBJECTIVES

PROJECT OBJECTIVES:

- To design and implement **combinational and sequential circuits** using Verilog/VHDL.
- To understand the concepts of **Hardware Description Language (HDL)** programming.
- To study and apply **behavioural modeling** techniques.
- To study and apply **structural modeling** techniques.
- To learn the working of basic digital building blocks such as gates , adders , flip-flops , registers, and counters.
- To perform **functional simulation** of digital circuits.
- To generate and analyze **simulation waveforms**.
- To verify the **logical correctness** of the designed circuits.
- To understand the role of **testbench** in digital circuit verification.
- To gain practical knowledge in **RTL design methodology**.
- To understand timing behavior using simulation tools.
- To develop debugging skills in digital circuit design.

TOOLS USED:

- **ALDEC RIVIERA-PRO**
- **VERILOG HDL**

**ALDEC RIVIERA WAS USED FOR WRITING RTL CODE ,
COMPILING , SIMULATING AND VERIFYING DIGITAL CIRCUITS .**

COMBINATIONAL AND SEQUENTIAL CIRCUITS

SEQUENTIAL CIRCUITS
AND
COMBINATIONAL

CIRCUIT SELECTION :

In this mini project, two digital circuits were selected to implement and verify using RTL modeling :

1. Combinational Circuit - 8-bit Arithmetic Logic Unit (ALU) :

An 8-bit ALU was selected as the combinational circuit .
The ALU performs arithmetic and logical operations such as :

- Addition
- Subtraction
- AND
- OR

The ALU is a combinational circuit because its output depends only on the present input values and does not require a clock signal or memory elements.

2. Sequential Circuit - 4-bit Register

A 4-bit register was selected as the sequential circuit .

The register stores 4-bit data and transfers it on the rising edge of the clock signal .

The register is a sequential circuit because its output depends on :

- Present input
- Previous stored value
- Clock signal
- It uses D Flip-Flops for storing data .

Reason for Selection :

The ALU and Register were selected to understand both :

- Combinational circuit design
- Sequential circuit design
- Difference between clock-based and non-clock-based circuits
- RTL modeling and simulation

This selection ensures proper understanding of digital design fundamentals and verification techniques.

THEORY OF COMBINATIONAL CIRCUIT :

8-BIT ALU (COMBINATIONAL CIRCUIT):

*ALU is a **COMBINATIONAL CIRCUIT** because ,
output depends on only present inputs .*

Arithmetic Logic Unit (ALU) :

Definition :

An **Arithmetic Logic Unit (ALU)** is a digital combinational circuit that performs arithmetic and logical operations on binary data based on control (selection) signals.
It is the main calculation unit inside the CPU.

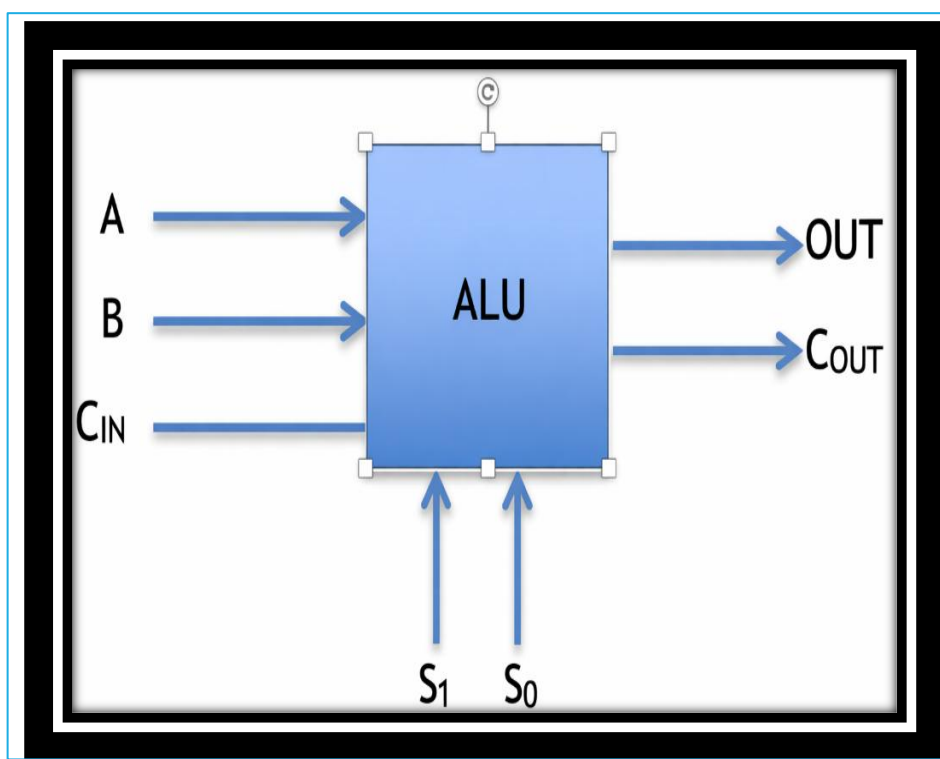
BLOCK DIAGRAM :

Inputs :

- A (8-bit operand)
- B (8-bit operand)
- Selection lines (S1, S0)

Outputs :

- F (8-bit result)
- Carry / Borrow flag



EXAMPLE : (IF ONLY 2 INPUTS)

Select Line	Operation	Description
00	Addition	Adds A and B
01	Subtraction	Subtracts B and A
10	AND	Bitwise AND
11	OR	Bitwise OR

1) ADDITION

$$F = A + B$$

Each bit is added using full adders.

Example:

$$A = 0101 \quad (5)$$

$$B = 0011 \quad (3)$$

$$F = 1000 \quad (8)$$

2) SUBTRACTION

$$F = A - B$$

Performed using 2's complement method:

$$A - B = A + (2's \text{ complement of } B)$$

3) AND OPERATION

$$F = A \cdot B$$

Bitwise multiplication

Example:

$$A = 1100$$

$$B = 1010$$

$$F = 1000$$

4) OR OPERATION

$$F = A + B$$

Bitwise addition (logical)

Example:

$$A = 1100$$

$$B = 1010$$

$$F = 1110$$

S.No	Behavioral Modeling	Structural Modeling
1	Describes what the circuit does	Describes how the circuit is built
2	High-level description	Low-level description
3	Uses always block	Uses module instantiation
4	Uses if, case, procedural statements	Uses gate connections and wires
5	Hardware structure is not visible	Hardware structure is clearly visible
6	Easier to write and understand	More complex to write
7	Suitable for RTL design	Suitable for gate-level design
8	Code looks like programming language	Code looks like circuit diagram
9	Faster to develop	Time-consuming to develop
10	Synthesis tool decides hardware implementation	Designer decides hardware connections

RTL CODE - BEHAVIOURAL MODULE

testbench.sv



```
1 `timescale 1ns/1ps
2
3 module alu_8bit_tb;
4
5 reg [7:0] A;
6 reg [7:0] B;
7 reg [1:0] sel;
8 wire [7:0] F;
9 wire carry;
10
11 // Instantiate ALU
12 alu_8bit uut (
13     .A(A),
14     .B(B),
15     .sel(sel),
16     .F(F),
17     .carry(carry)
18 );
19
20 initial
21 begin
22     // Dump file for waveform
23     $dumpfile("alu.vcd");
24     $dumpvars(0, alu_8bit_tb);
25
26     // Test cases
27     A = 8'd10; B = 8'd5; sel = 2'b00; #10;
28     A = 8'd10; B = 8'd3; sel = 2'b01; #10;
29     A = 8'b11001100; B = 8'b10101010; sel = 2'b10; #10;
30     A = 8'b11001100; B = 8'b10101010; sel = 2'b11; #10;
31
32     $finish;
33 end
34
35 endmodule
```

design.sv



```
1 // 8-bit ALU - Behavioral Model
2 module alu_8bit (
3     input [7:0] A,
4     input [7:0] B,
5     input [1:0] sel,
6     output reg [7:0] F,
7     output reg carry
8 );
9
10 always @(*) // Combinational always block
11 begin
12     case(sel)
13         2'b00: begin
14             {carry, F} = A + B; // Addition
15         end
16
17         2'b01: begin
18             {carry, F} = A - B; // Subtraction
19         end
20
21         2'b10: begin
22             F = A & B; // AND
23             carry = 0;
24         end
25
26         2'b11: begin
27             F = A | B; // OR
28             carry = 0;
29         end
30         |
31         default: begin
32             F = 8'b00000000;
33             carry = 0;
34         end
35     endcase
36 end
37 endmodule
```

RTL CODE - STRUCTURAL MODULE

testbench.sv



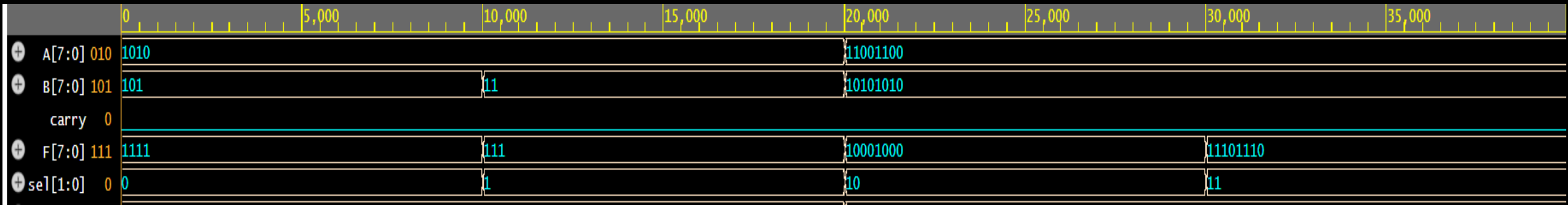
```
1 `timescale 1ns/1ps
2
3 module alu_structural_tb;
4
5 reg [7:0] A;
6 reg [7:0] B;
7 reg [1:0] sel;
8 wire [7:0] F;
9 wire carry;
10
11 // Instantiate ALU
12 alu_structural uut (
13     .A(A),
14     .B(B),
15     .sel(sel),
16     .F(F),
17     .carry(carry)
18 );
19
20 initial
21 begin
22     $dumpfile("alu_structural.vcd");
23     $dumpvars(0, alu_structural_tb);
24
25     // ADD
26     A = 8'd10; B = 8'd5; sel = 2'b00; #10;
27
28     // SUB
29     A = 8'd10; B = 8'd3; sel = 2'b01; #10;
30
31     // AND
32     A = 8'b11001100; B = 8'b10101010; sel = 2'b10; #10;
33
34     // OR
35     A = 8'b11001100; B = 8'b10101010; sel = 2'b11; #10;
36
37     $finish;
38 end
39
40 endmodule
```

design.sv



```
1 // 8-bit ALU - Structural (Single Module)
2 module alu_structural (
3     input [7:0] A,
4     input [7:0] B,
5     input [1:0] sel,
6     output [7:0] F,
7     output carry
8 );
9 // Internal wires
10 wire [7:0] add_out;
11 wire [7:0] sub_out;
12 wire [7:0] and_out;
13 wire [7:0] or_out;
14 wire [7:0] B_comp;
15 wire [8:0] add_temp;
16 wire [8:0] sub_temp;
17 // ----- ADDITION -----
18 assign add_temp = A + B;
19 assign add_out = add_temp[7:0];
20 // ----- SUBTRACTION -----
21 assign B_comp = ~B + 1; // 2's complement
22 assign sub_temp = A + B_comp;
23 assign sub_out = sub_temp[7:0];
24 // ----- LOGICAL -----
25 assign and_out = A & B;
26 assign or_out = A | B;
27 // ----- 4-to-1 MUX -----
28 assign F = (sel == 2'b00) ? add_out :
29           (sel == 2'b01) ? sub_out :
30           (sel == 2'b10) ? and_out :
31           or_out;
32 assign carry = (sel == 2'b00) ? add_temp[8] :
33               (sel == 2'b01) ? sub_temp[8] :
34               1'b0;
35
36 endmodule
```

WAVEFORM RESULTS



If:

- sel = 00 → Addition
- sel = 01 → Subtraction
- sel = 10 → AND
- sel = 11 → OR

EXPLANATION OF ALWAYS BLOCK

What is `always` block?

`always` block is used in Verilog to describe **behavioral modeling**.

It executes continuously whenever there is a change in the sensitivity list.

`always @(*)`

- `@(*)` means: execute whenever any input changes
- Used for **combinational circuits**
- No clock required

It ensures:

- Output changes immediately when input changes
- No memory element is created

Why `reg` used in output?

Inside `always` block:

- Output must be declared as `reg`
- Because values are assigned procedurally

Example:

```
output reg [7:0] F;
```

EXPLANATION OF CASE STATEMENT

What is case?

case is used to select one operation among many based on selection input. It works like a switch statement in C language.

Structure

```
case (expression)
    value1: statement1;
    value2: statement2;
    default: statement;
endcase
```

In ALU

```
case (sel)
```

lf:

- sel = 00 → Addition
- sel = 01 → Subtraction
- sel = 10 → AND
- sel = 11 → OR

THEORY OF SEQUENTIAL CIRCUIT

1. Definition of Sequential Circuit

A **Sequential Circuit** is a type of digital circuit whose output depends on:

- **Present input**
- **Previous output (past state)**

Unlike **combinational circuits**, sequential circuits have **memory**.
The memory element is usually a **Flip-Flop**.

2. What is a 4-Bit Register?

A **4-bit Register** is a sequential circuit that can:

- **Store 4 bits of binary data**
- **Hold the data until the next clock pulse updates it**

It is made using:

- **4 D Flip-Flops**
- **A common Clock signal**

Each flip-flop stores **1 bit**, so:

4 Flip-Flops $\Rightarrow$ 4 bits storage

3. Use of Clock in 4-Bit Register

The **Clock** is a periodic square wave signal.

Purpose of Clock:

- Controls when data is stored
- Makes all flip-flops update **simultaneously**
- Ensures synchronous operation

How it works:

- On the **rising edge** (↑) of the clock,
- The input data is transferred to the output.

Without clock → No synchronization

With clock → Controlled data storage

4. D Flip-Flop in 4-Bit Register

A D Flip-Flop is a memory element.

Function: $Q_{next} = D$

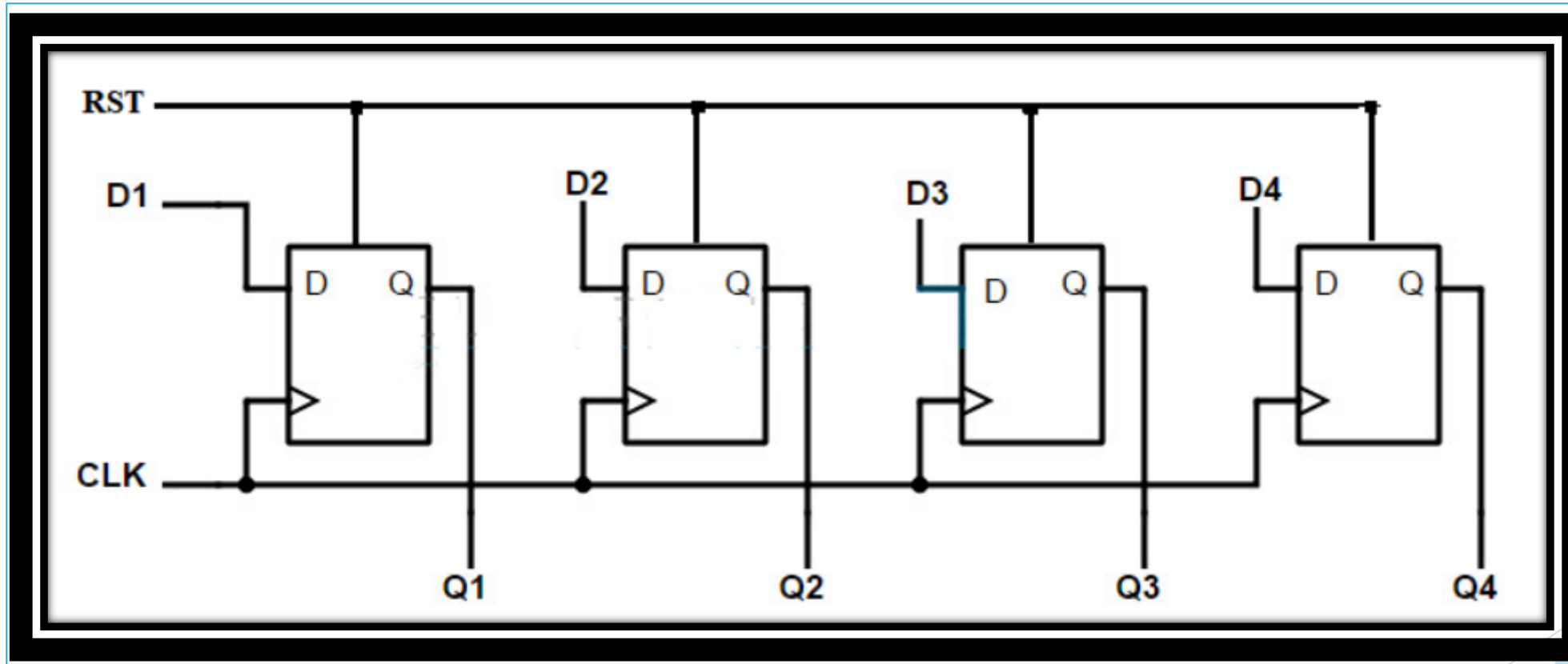
Meaning:

- Output Q takes the value of input D
- Only at the active clock edge

TRUTH TABLE :

CLOCK EDGE	D	Q(Next)
1	0	0
1	1	1
NO EDGE	X	NO CHANGE

BLOCK DIAGRAM OF 4 BIT REGISTER



RTL CODE – REGISTER(4BIT)

RTL CODE – REGISTER(4BIT)

**BEHAVIOURAL
AND
STRUCTURAL
CODES**

CODES
STRUCTURAL
AND
BEHAVIOURAL

testbench.sv



```
1 `timescale 1ns/1ps
2 module register_4bit_tb;
3
4 reg clk;
5 reg reset;
6 reg [3:0] d;
7 wire [3:0] q;
8
9 // Instantiate the Design Under Test (DUT)
10 register_4bit uut (
11     .clk(clk),
12     .reset(reset),
13     .d(d),
14     .q(q)
15 );
16
17 // Clock Generation (10ns period)
18 always #5 clk = ~clk;
19
20 initial
21 begin
22     // Dump file for waveform
23     $dumpfile("register_4bit.vcd");
24     $dumpvars(0, register_4bit_tb);
25
26     // Initialize signals
27     clk = 0;
28     reset = 1;
29     d = 4'b0000;
30
31     #10 reset = 0;    // Release reset
32
33     #10 d = 4'b1010; // Apply first input
34     #10 d = 4'b1100; // Apply second input
35     #10 d = 4'b1111; // Apply third input
36     #10 d = 4'b0011; // Apply fourth input
37
38     #20 $finish;    // End simulation
39 end
40 endmodule
41
```

design.sv



```
1 module register_4bit (
2     input clk,
3     input reset,
4     input [3:0] d,
5     output reg [3:0] q
6 );
7
8 always @(posedge clk)
9 begin
10     if (reset)
11         q <= 4'b0000;
12     else
13         q <= d;
14 end
15
16 endmodule
17
```



```

1 `timescale 1ns/1ps
2 module register_4bit_structural_tb;
3
4 reg clk;
5 reg reset;
6 reg [3:0] d;
7 wire [3:0] q;
8
9 // Instantiate DUT
10 register_4bit_structural uut (
11     .clk(clk),
12     .reset(reset),
13     .d(d),
14     .q(q)
15 );
16
17 // Clock Generation (10ns period)
18 always #5 clk = ~clk;
19
20 initial
21 begin
22     // Dump file for waveform
23     $dumpfile("register_4bit_structural.vcd");
24     $dumpvars(0,
25         register_4bit_structural_tb);
26
27     // Initialize signals
28     clk = 0;
29     reset = 1;
30     d = 4'b0000;
31
32     #10 reset = 0;           // Release reset
33     #10 d = 4'b1010;
34     #10 d = 4'b1100;
35     #10 d = 4'b1111;
36     #10 d = 4'b0011;
37
38     #20 $finish;
39 end
40 endmodule

```

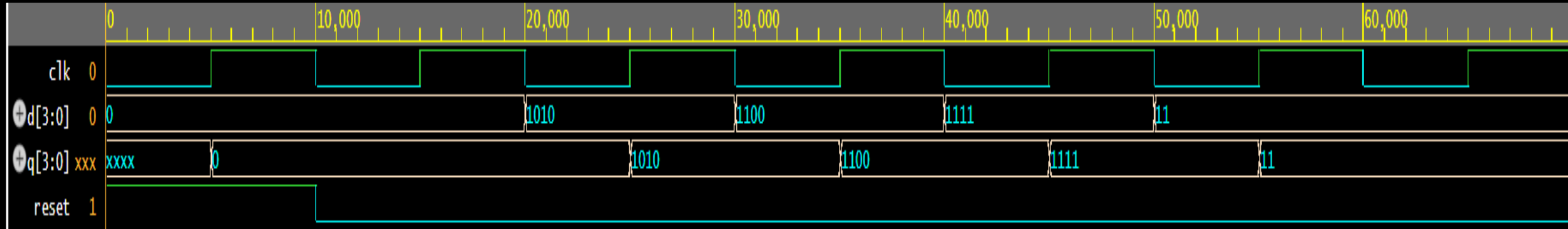


```

1 // =====
2 // D Flip-Flop Module
3 // =====
4 module d_ff (
5     input clk,
6     input reset,
7     input d,
8     output reg q
9 );
10
11 always @(posedge clk)
12 begin
13     if (reset)
14         q <= 1'b0;
15     else
16         q <= d;
17 end
18
19 endmodule
20
21 // =====
22 // 4-Bit Register (Structural)
23 // =====
24 module register_4bit_structural (
25     input clk,
26     input reset,
27     input [3:0] d,
28     output [3:0] q
29 );
30
31 // Instantiate 4 D Flip-Flops
32 d_ff ff0 (.clk(clk), .reset(reset), .d(d[0]), .q(q[0]));
33 d_ff ff1 (.clk(clk), .reset(reset), .d(d[1]), .q(q[1]));
34 d_ff ff2 (.clk(clk), .reset(reset), .d(d[2]), .q(q[2]));
35 d_ff ff3 (.clk(clk), .reset(reset), .d(d[3]), .q(q[3]));
36
37 endmodule
38
39

```

WAVEFORM RESULTS



SIGNAL	MEANING
clk	Clock signal
reset	Clears register
d[3:0]	Input data
q[3:0]	Stored output

FEATURE	8-BIT ALU	4-BIT REGISTER
Type	Combinational	Sequential
Clock	No	Yes
Memory	No	Yes
Output change	Immediately	Only at posedge clk
Reset	Usually no	Yes

DEBUGGING AND VERIFICATION

VERIFICATION
AND
DEBUGGING

WHAT IS DEBUGGING?

Debugging is the process of:

Identifying, analyzing, and correcting errors in a Verilog design or testbench.

It is done during **simulation and compilation**.

WHAT IS VERIFICATION?

Verification means:

Checking whether the RTL design works according to the expected functionality.

It is done using:

- Testbench
- Simulation
- Waveform analysis
- Test vectors

A) SYNTAX ERRORS

Meaning : Errors due to incorrect Verilog writing rules.

ERRORS	REASON	CORRECTION
Missing semicolon ;	Forgot ; after statement	Added semicolon
Wrong module name	Module name mismatch in testbench	Corrected module name
Missing endmodule	Block not closed	Added endmodule
Wrong port connection	Signal name mismatch	Corrected signal mapping
Forgot reg in testbench	Inputs must be reg	Declared properly

B) LOGICAL ERRORS

Meaning : Code compiles successfully but gives wrong output .

Error	Reason	Correction
Using = instead of <=	Blocking used in sequential logic	Replaced with non-blocking <=
Reset not working	Reset not inside always block	Added proper reset condition
ALU wrong operation	Incorrect case statement	Corrected case conditions
Missing default case	Output became X	Added default case

C) SIMULATION ERRORS

Errors Faced:

No clock generation
Forgot `$finish`
Dump file not created
Signals showing x (unknown state)

Corrections:

Added clock generator
Added `$dumpfile`
Initialized signals
Added reset condition

➤ **DEBUGGING STEPS FOLLOWED**

- 1.Compiled using ALDEC RIVERA
- 2.Observed error messages
- 3.Corrected syntax mistakes
- 4.Recompiled
- 5.Ran simulation
- 6.Observed waveform
- 7.Compared expected vs actual output
- 8.Fixed logical mistakes
- 9.Recompiled again
- 10.Verified correct waveform

➤ **SUCCESSFUL RECOMPILATION**

After correcting:

- Syntax errors removed
- Logical errors corrected
- Waveform matched expected results
- No warnings during compilation
- Simulation completed successfully

❖ RESULTS ACHIEVED

- ❖ SUCCESSFULLY IMPLEMENTED ALU AND REGISTER
- ❖ RTL CODE COMPILED WITHOUT ERRORS
- ❖ FUNCTIONAL SIMULATION COMPLETED
- ❖ OUTPUTS MATCHED EXPECTED RESULTS

VERIFICATION COMPLETED SUCCESSFULLY

The background features abstract, overlapping geometric shapes in various shades of blue, primarily on the right side of the frame. The shapes include triangles and polygons, creating a modern, layered effect. The left side of the image is mostly white, providing a clean backdrop for the text.

CONCLUSION

CONCLUSION

- ✓ LEARNED RTL MODELING
- ✓ UNDERSTOOD BEHAVIORAL V/S STRUCTURAL MODELING
- ✓ GAINED KNOWLEDGE IN SIMULATION AND WAVEFORM ANALYSIS
- ✓ IMPROVED DEBUGGING SKILL



THE END

